Guidelines for Implementation and Code Design

1. Use object-oriented programming. For Q1, all sampling and analysis must use the original dataset stored in the `StudentDataset` object.
2. Use appropriate visualization libraries such as `matplotlib`, `seaborn`, or `plotly`. Each plot must include a **title, x-label, y-label, and legend (if applicable)**. Answers with missing labels or legends will receive **0 marks**.
3. Keep visualization and computation logic in **separate functions**.
4. Add `docstrings` to methods explaining what they do.
5. Use reproducible random sampling using the given `seed`.

Q1.0 Dataset Generation [6 marks]

Generate 10,000 student records with the following attributes:

- **gender:** Male (65%), Female (33%), Other (2%) [1]
- **major:** B.Tech (70%), MS (20%), PhD (10%) [1]
- **program:** distribution conditioned on major: [2]

Major	CSE	ECE	CHD	CND
B.Tech	40%	40%	10%	10%
MS	30%	30%	20%	20%
PhD	25%	25%	25%	25%

- **GPA:** Normally distributed by major, clipped to [4.0, 10.0] [2]

Major	GPA Distribution ($\mathcal{N}(\mu, \sigma)$)
B.Tech	$\mathcal{N}(7.0, 1.0)$
MS	$\mathcal{N}(8.0, 0.7)$
PhD	$\mathcal{N}(8.3, 0.5)$

You must implement a class as follows.

```
class StudentDataset:
    def __init__(self, num_students: int, seed: int):
    # Generates the full dataset during initialization using the
       specified number of students and seed.
    def get_full_dataframe(self) -> pd.DataFrame:
    # Do not regenerate the dataset in different methods or cells.
    # Use this method to access the full dataset consistently.
```

```
    def generate_gender(self) -> list[str]: ...
    def generate_major(self) -> list[str]: ...
    def generate_program(self, majors: list[str]) -> list[str]: ...
    def generate_gpa(self, majors: list[str]) -> list[float]: ...
    def assemble_dataframe(self) -> pd.DataFrame: ... # Assemble the
        full dataset from gender, major, program, and GPA.
```

Q1.1 Dataset Analysis

(a) Visualizations [15 marks]

Create suitable visualizations for the following distributions. The visualizations should convey meaningful information about the data.

- gender [1]
- major [1]
- program [1]
- GPA [1]
- program conditioned on major [1]
- GPA conditioned on major [1]
- GPA conditioned on program [1]
- GPA conditioned on program and major [2]
- gender, major, program and GPA of 100 randomly sampled students [3]
- Summary of entire dataset(e.g. pairplots) [3]

You may implement the following methods:

```
def plot_gender_distribution(self) -> None: ...
def plot_major_distribution(self) -> None: ...
def plot_program_distribution(self) -> None: ...
def plot_gpa_distribution(self, bins: int = 20) -> None: ...
def plot_program_by_major(self) -> None: ...
def plot_gpa_by_major(self) -> None: ...
def plot_gpa_by_program(self) -> None: ...
def plot_gpa_by_program_and_major(self) -> None: ...
def plot_sampled_dataset(self) -> None: ...
def plot_entire_dataset_summary(self) -> None: ...
```

(b) GPA Summary Statistics [1 mark]

Define a method to compute the mean and standard deviation of GPA:

```
def gpa_mean_std(self) -> tuple[float, float]: ...
```

Report the results and briefly comment on any observations.

(c) Program-Major Combinations [2 marks]

Define a method to count the number of students for each unique (program, major) pairs. Also write a method to visualize it with a heatmap.

```
def count_students_per_program_major_pair(self) -> pd.Dataframe: ...
def visualize_students_per_program_major_pair(self, counts_df : pd.
   Dataframe) -> None: ...
```

Report the counts and describe any patterns you observe.

Q1.2 Simple vs Stratified Sampling [5 marks]

- Sample 500 students uniformly at random. Repeat 50 times. Calculate the mean GPA and standard deviation for each sample. Report the average mean GPA and the standard deviation of the mean GPAs across the 50 repetitions. [2]
- Repeat using stratified sampling by major (divide population into strata by major based on their proportion in the dataset). Compare the results. [2]
- Which method has lower std deviation? Why? [1]

```
def get_gpa_mean_std_random(self, n: int = 500, repeats: int = 50)
   -> tuple[float, float]: ...
def get_gpa_mean_std_stratified(self, n: int = 500, repeats: int =
   50) -> tuple[float, float]: ...
```

Q1.3 Gender-Balanced Cohort [5 marks]

- Sample 300 students with exact same count across genders. Repeat 5 times. Report gender counts. [1]
- Consider the following **Sampling Strategy A**: Randomly pick a value from a discrete set of categories with equal probability (here, gender). Randomly pick a student from that category. Sample 300 students using this sampling strategy. Repeat 5 times. Report gender counts. [2]

- For each sample size (300, 600, 900, 1200, 1500), repeat the above sampling process 10 times. In each repeat, count the number of students in each gender, find the difference between the largest and smallest counts, and divide by the sample size (n) to get the maximum relative difference. Compute the average of these values over the 10 repeats for each sample size. Plot a histogram of average maximum relative difference (y-axis) versus sample size (x-axis). [2]

```
def get_gender_balanced_counts(self, n: int = 300, repeats: int = 5)
    -> list[dict[str, int]]: ...
def sample_gender_uniform_random(self, n: int = 300, repeats: int =
   5) -> list[dict[str, int]]:
def plot_avg_max_gender_diff_vs_sample_size(self, sample_sizes: list
   [int], repeats: int = 10) -> None: ...
```

Q1.4 GPA-Uniform Cohort [3 marks]

- Using Sampling Strategy A, select 100 students such that their GPA values are approximately uniformly distributed across 10 bins. [1]
- Plot GPA histogram and compare to original dataset's histogram. [1]
- Did you sample with or without replacement? Why? [1]

```
def sample_gpa_uniform(self, n: int = 100, bins: int = 10) -> pd.
   DataFrame: ...
def plot_gpa_histogram_comparison(self, sampled_df: pd.DataFrame) ->
    None: ...
```

Q1.5 Program-Major Balanced Cohort [3 marks]

- Using Sampling Strategy A, select 60 students such that all valid (program, major) combinations are represented approximately equally. [1]
- Show counts and heatmap. [1]
- Were any groups too small? How did you handle it? [1]

```
def sample_program_major_balanced(self, n: int) -> pd.DataFrame: ...
def show_program_major_counts_and_heatmap(self, sampled_df: pd.
   DataFrame) -> None: ...
```